Database Users, Roles, Privileges, and Access Control

Group 4 (BrewPOS - Cafe Point-of-Sale System)

Section	Contents
Cover and links	**GROUP 12: Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix** **Project Assigned:** BrewPOS - Cafe Point-of-Sale System **Repository Link: https://github.com/Nycri/brewpos** **Our Group Repository:**
Projects set up	**The BrewPOS repository was used as the basis for implementing the database access-control plan. The project is a Point-of-Sale (POS) application designed to manage coffee shop operations, including products, orders, sales, and other related business information.** **The first step was to examine the existing project files and understand how the application connects to its database. The database configuration and existing SQL files were reviewed to identify the database name, tables, and other database objects that needed access restrictions.** **The database used for this activity was named [INSERT ACTUAL DATABASE NAME]. The existing database structure was retained, and the access-control implementation was performed separately to avoid changing the original system functionality.** **The following database tables were identified:** • **[ACTUAL TABLE NAME 1] – Stores information about [describe its purpose].** • **[ACTUAL TABLE NAME 2] – Stores information about [describe its purpose].** • **[ACTUAL TABLE NAME 3] – Stores information about [describe its purpose].** • **[ACTUAL TABLE NAME 4] – Stores information about [describe its purpose].** **After reviewing the database structure, a separate folder named `access_control` was prepared to organize the SQL scripts and access-control documentation.**

	The following files were created: • `01_create_roles.sql` • `02_grant_privileges.sql` • `03_revoke_privileges.sql` • `04_verify_permissions.sql` • `access_control_plan.md` **These files were organized to make the implementation easier to understand, execute, and maintain.**
Access-control plan	**The access-control plan was created to determine which database operations should be allowed for each role. The implementation follows the principle of least privilege, meaning that each role should only receive the permissions necessary for its responsibilities.** **Three PostgreSQL group roles were established.** **Administrator Role (`brewpos_admin_grp`)** **The administrator role is responsible for managing and maintaining the database. It receives the privileges required to manage the database objects and records within the assigned scope.** **Its responsibilities include maintaining product information, managing sales records, handling order information, and maintaining other important database records.** **Runtime Role (`brewpos_runtime_grp`)** **The runtime role is intended for normal application operations. It allows the application to retrieve, add, update, and delete database records when performing its functions.** **Its responsibilities include retrieving product information, recording customer orders, processing sales transactions, and updating records required by the application.** **Read-only Role (`brewpos_readonly_grp`)** **The read-only role is intended for users who only need to view database information. It receives SELECT privileges but cannot insert, update, or**

	delete records. **Its purpose is to allow authorized users to view business information without making unauthorized changes.** **Access-Control Matrix** **Database Operations** / **Administrator** / **Runtime** / **Read-only** **SELECT** / **Allowed** / **Allowed** / **Allowed** **INSERT** / **Allowed** / **Allowed** / **Denied** **UPDATE** / **Allowed** / **Allowed** / **Denied** **DELETE** / **Allowed** / **Allowed** / **Denied** **Sequence Access** / **Full (as needed)** / **Usage and Select (as needed)** / **Not granted unless required** **The three roles were created with the `NOLOGIN` attribute. This means they function as permission groups and must be assigned to actual login accounts before individual users or application accounts can use their permissions.**
SQL scripts summary	**Three main SQL implementation scripts and a verification script were prepared to implement and check the access-control configuration.** **File 1: `01_create_roles.sql`** **The first SQL script was used to create the three PostgreSQL group roles.** **The script checks whether each role already exists before creating it. This helps prevent errors when the script is executed more than once.** **The following roles were created:** • **`brewpos_admin_grp`** • **`brewpos_runtime_grp`** • **`brewpos_readonly_grp`** **After executing the script through pgAdmin 4, a verification query was**

	used to display the created roles from the PostgreSQL system catalog. **File 2: `02_grant_privileges.sql`** **The second SQL script was used to assign the required database permissions to each role.** **First, the three roles were given permission to connect to the BrewPOS database and access the required schema.** **The administrator role received the required administrative privileges on the existing database objects. The runtime role received SELECT, INSERT, UPDATE, and DELETE permissions on the tables needed by the application. The read-only role received SELECT privileges only on the approved tables and views.** **The required sequence permissions were also assigned where necessary for inserting records that use sequence-generated identifiers.** **File 3: `03_revoke_privileges.sql`** **The third SQL script was used to remove privileges that should not be available to restricted roles.** **The read-only role was restricted from performing INSERT, UPDATE, and DELETE operations. This ensures that users assigned to this role can only view authorized database information.** **The script helps maintain the principle of least privilege by removing unnecessary access rights.** **File 4: `04_verify_permissions.sql`** **The fourth SQL script was used to check whether the assigned permissions were correct.** **The script contains queries that display the created roles, list their table privileges, and test specific permissions using PostgreSQL's `has_table_privilege()` function.** **The verification focused on checking whether:** **• The administrator can perform authorized database operations.** **• The runtime role can insert records.** **• The runtime role can update records.** **• The read-only role can select records.**

	• **The read-only role cannot insert records.** • **The read-only role cannot update or delete records.** **These checks help confirm that each role follows the intended access-control plan.**
Verification results	**After preparing the SQL scripts, they were executed in the PostgreSQL Query Tool through pgAdmin 4.** **The scripts were executed in the correct order to avoid permission-related errors.** **First Verification: Role Creation** **The first script was executed to create the three group roles. The PostgreSQL system catalog was queried to confirm that the roles existed.** **Expected result: All three group roles should appear in the query results.** **Second Verification: Privilege Assignment** **The second script was executed to assign the required database, schema, table, and sequence privileges.** **The execution messages were checked to determine whether the commands completed successfully.** **Expected result: Each role should receive the permissions defined in the access-control plan.** **Third Verification: Privilege Restrictions** **The third script was executed to remove unnecessary privileges from restricted roles.** **Expected result: The read-only role should not have permission to modify database records.** **Fourth Verification: Permission Checking** **The fourth script was executed to verify the permissions using PostgreSQL queries.**

<table>
<tr>
<td></td>
<td>
The expected results were:
<table>
<tr><th>Verification</th><th>Expected Result</th></tr>
<tr><td>Administrator can perform authorized operations</td><td>TRUE</td></tr>
<tr><td>Runtime can insert records</td><td>TRUE</td></tr>
<tr><td>Runtime can update records</td><td>TRUE</td></tr>
<tr><td>Read-only can select records</td><td>TRUE</td></tr>
<tr><td>Read-only cannot insert records</td><td>FALSE</td></tr>
<tr><td>Read-only cannot update records</td><td>FALSE</td></tr>
<tr><td>Read-only cannot delete records</td><td>FALSE</td></tr>
</table>
The expected results demonstrate that the administrator and runtime roles have their intended database permissions, while the read-only role is restricted to viewing records.

Note: These are expected results, not confirmed execution results. Actual results must be recorded after running the scripts and testing the permissions using the assigned roles.

<h3>Screenshots</h3>
The following screenshots should be included as supporting evidence:

Screenshot 1: <code>G4_create_roles.png</code>
Screenshot 2: <code>G4_grant_privileges.png</code>
Screenshot 3: <code>G4_revoke_privileges.png</code>
Screenshot 4: <code>G4_test_privileges.png</code>
Screenshot 4: <code>G4_check_role_membership.png</code>

The screenshots serve as supporting evidence of the SQL execution and verification process.
</td>
</tr>
<tr>
<td>Issues and fixes</td>
<td>

During the preparation and implementation of the access-control configuration, the following possible issues were considered and addressed.

</td>
</tr>
</table>

	● Database Name Mismatch: The SQL scripts must be executed inside the correct database. The actual BrewPOS database name must be used consistently to avoid applying the configuration to the wrong database. **● Existing Database Roles: Running role creation commands repeatedly may cause errors if the roles already exist. The first SQL script uses an existence check before creating each role.** **● Incorrect Execution Order: Granting permissions before creating the roles can result in errors. To prevent this, the scripts were executed in order, starting with role creation, followed by privilege assignment, privilege restrictions, and verification.** **● Insufficient Database Permissions: Some PostgreSQL commands require appropriate administrative privileges. The scripts should be executed using an authorized PostgreSQL administrator account.** **● Incorrect Table Names: Granting privileges to tables that do not exist can result in relation errors. To prevent this, the actual table names were checked using PostgreSQL database inspection queries before writing the GRANT statements.** **● Future Database Objects: Newly created tables and sequences may not automatically receive the same permissions as existing objects. Default privileges should be configured for the correct object owner when necessary.** **● Read-only Permission Restrictions: The read-only role should not be able to modify database records. The verification script checks that SELECT is allowed while INSERT, UPDATE, and DELETE are denied.**
Feedback to owners	**The BrewPOS application provides database functionality for managing point-of-sale operations. However, implementing role-based access control can further improve the security and organization of database operations.** **The following recommendations are provided to the project owners:** **● Maintain separate database roles for administrators and normal application operations.** **● Avoid using administrator-level database credentials for regular application activities.** **● Assign actual login accounts to the appropriate group roles.** **● Use strong passwords and protect database credentials.** **● Review database permissions whenever new tables or features are added.** **● Restrict access to sensitive sales, payment, and user information.**

	• **Regularly check database roles and privileges to prevent unnecessary access.** • **Use read-only accounts for users who only need to view business information.** **These recommendations can help improve the security and maintainability of the BrewPOS application.**
AI usage declaration	**Artificial Intelligence (AI) tools were used as a supplementary resource during the activity to assist in understanding PostgreSQL role creation, database privileges, SQL syntax, and documentation preparation.** **AI assistance was used to help organize the SQL scripts, explain the purpose of each access-control role, and identify possible implementation issues and their corresponding solutions.** **The SQL scripts were reviewed and executed using PostgreSQL through pgAdmin 4. The student remains responsible for checking the correctness of the implementation, understanding the assigned permissions, and verifying the actual execution results.**